HTML & CSS Handbook — Interview Edition

How to use this edition

This is the night-before-the-interview version of the *HTML & CSS Handbook*: every topic from the 14 chapters, compressed into facts, numbers, tiny snippets, likely questions and traps. It adds nothing that isn't in the full book; when an answer here feels thin, open the matching chapter in `../chapters/` for the explanation, the measured browser output and the figures.

- **One section per chapter**, always in the same order: **Must-know** (one-line definitions) → **Key facts & numbers** (tables, formulas, thresholds, support status) → **Code to remember** → **In the interview** (Q/A you can say out loud) → **Gotchas** → **TL;DR**.
- **Numbers are the book's own**: many were measured in headless Chrome 154 (flex/grid sizes, contrast ratios, Web Vitals). Pixel values can differ slightly on other machines; the pattern won't.
- **Baseline labels** come from web-features / MDN browser-compat-data, checked September 2026. "Newly available" = supported in all core browsers; "widely available" = 30 months after that.
- **How to revise**: (1) read the Top 25 below and answer each before looking; (2) skim the Must-know and TL;DR of every chapter; (3) for the classics — specificity and the cascade (Ch 6), margin collapsing, BFCs and stacking contexts (Ch 7), `flex: 1` and `min-width: 0` (Ch 8), `fr` and `auto-fill` / `auto-fit` (Ch 9), Core Web Vitals (Ch 14) — work the numbers by hand; (4) finish with the Last-minute checklist.
- **The five formulas to know cold**: specificity (A, B, C) with no carrying; the cascade order; flex grow `basis + free × grow/Σgrow` and shrink weighted by `shrink × basis`; `fr` = (container − fixed tracks − gaps) / Σfr; contrast `(L1 + 0.05)/(L2 + 0.05)`.

Top 25 questions across the book

#	Question	One-line answer	Ch
1	What happens when you type a URL and press Enter?	DNS → TCP+TLS (or QUIC) → HTTP GET → HTML parsed into DOM, CSS into CSSOM → render tree → layout → paint → composite.	1
2	`defer` vs `async`?	Both download in parallel; `defer` runs in order after parsing, `async` runs on arrival in any order.	1, 14
3	What does the doctype do?	Standards mode; without it, quirks mode.	2
4	What is the first rule of ARIA?	Use native HTML first; ARIA changes semantics only, adds no behaviour.	3
5	Link vs button?	Links navigate (`href`), buttons act on the page; a `div` with `onclick` is neither focusable nor announced.	2, 3
6	WCAG AA contrast?	4.5:1 normal text, 3:1 large text (≥24px or ≥18.66px bold) and UI components.	3, 11
7	How do `srcset` and `sizes` work?	`srcset` lists candidates (`w`), `sizes` gives the display width before CSS loads; the browser picks by DPR. `<picture>` when you must choose.	4
8	What does a form submit?	Named, enabled, checked controls to `action` via `method` ; GET for reads, POST for changes, multipart for files.	5
9	How is specificity calculated?	(IDs, classes/attrs/pseudo-classes, types/pseudo-elements), compared left to right, no carrying; `:where` = 0; `:is/:not/:has` = max argument.	6
10	What decides which declaration wins?	Origin & importance → context → inline style → layers → specificity → (scope proximity) → order.	6
11	`em` vs `rem`?	`em` = element's font size (parent's in `font-size`, compounds); `rem` = root.	6
12	Explain the box model and `box-sizing`.	Content+padding+border+margin; content-box adds padding/border to `width` (200 → 252), border-box includes them.	7
13	What is margin collapsing?	Adjacent vertical margins in block flow merge to the largest; not in flex/grid, floats, absolute, or across padding/border/BFC.	7
14	What is a BFC?	Independent layout region (`display: flow-root`) that contains floats and stops margin collapse.	7
15	Why doesn't my `z-index` work?	Only positioned/flex/grid items, and only within the same stacking context (opacity < 1, transform, etc. create one).	7
16	Explain the `position` values.	relative keeps space; absolute → nearest positioned ancestor; fixed → viewport (unless transformed ancestor); sticky → threshold in scroll container.	7
17	What does `flex: 1` mean?	`1 1 0%` — share all space equally from a zero basis.	8
18	Why does a flex item overflow instead of shrinking?	`min-width: auto` = min-content; set `min-width: 0`.	8
19	How do you centre something both ways?	`display: grid; place-items: center` (or flex + `justify-content` + `align-items: center`).	8, 9
20	Flexbox or grid?	Flexbox: 1D, content-out; grid: 2D, layout-in; nest them.	9
21	`auto-fill` vs `auto-fit`?	Both make as many tracks as fit; auto-fit collapses empty tracks so items stretch.	9

#	Question	One-line answer	Ch
22	Media vs container queries?	Viewport vs a declared ancestor's size; page layout vs reusable components.	10
23	Which properties are cheap to animate?	`transform` and `opacity` — composite only, no layout or paint.	12
24	Core Web Vitals and thresholds?	LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 at p75; INP replaced FID March 2024.	14
25	Dark mode with a user toggle?	Semantic tokens with `light-dark()`, `color-scheme: light dark`, `[data-theme]` pins `color-scheme`, saved in localStorage, applied in `<head>` before first paint.	13

1. How the Web Works

Clients and servers, URLs, DNS, TCP/TLS/HTTP versions, requests and responses, "type a URL", the rendering pipeline.

Must-know

- **Client / server** — the client (browser, app, `curl`) asks; the server answers. The **user agent** is the spec's name for the browser acting for the user.
- **Engines** — browser engines: **Blink** (Chrome, Edge, Opera), **Gecko** (Firefox), **WebKit** (Safari). JS engines: V8, SpiderMonkey, JavaScriptCore.
- **HTML / CSS / JS** — structure and meaning / presentation / behaviour.
- **URL** — `scheme://host:port/path?query#fragment`. The **fragment** is never sent to the server. Default ports: 80 (http), 443 (https).
- **Origin** — scheme + host + port.
- **Relative URLs** — resolve against the current page's URL; `.` = this folder, `..` = parent; root-relative `/cart` starts at the site root.
- **DNS** — resolves a name to an IP through resolver → root → TLD → authoritative servers, cached per TTL. Classic DNS is UDP port 53, unencrypted; **DoH/DoT** encrypt it.
- **HTTPS** — TCP + TLS; gives confidentiality, integrity and authentication. **HSTS** (`Strict-Transport-Security`) makes the browser upgrade to HTTPS itself.
- **HTTP** — stateless request/response: method + path + headers → status + headers + body. HTML alone can send only **GET and POST**.
- **Rendering pipeline** — HTML → **DOM**, CSS → **CSSOM** → **render tree** → **layout** → **paint** → **composite**.
- **Blocking** — CSS is **render-blocking**; classic scripts are **parser-blocking**; `defer` / modules fix it.
- **Reflow** — recomputing layout because geometry changed; expensive. **Layout thrashing** = read a layout value (e.g. `offsetHeight`) after each style write in a loop.

Key facts & numbers

Version	Transport	Key idea	Effect
HTTP/1.1	TCP	Text, persistent connections	One request at a time per connection; ~6 connections per host
HTTP/2	TCP	Binary framing, **multiplexing**, header compression	Many small files cheap; one lost packet still stalls all streams (TCP head-of-line blocking)
HTTP/3	**QUIC** over UDP	Independent streams, TLS 1.3 built in	Faster set-up, better on lossy mobile, survives network changes

Method	Safe	Idempotent
GET, HEAD, OPTIONS	Yes	Yes
PUT, DELETE	No	Yes
POST	No	No
PATCH	No	Not necessarily

Status	Know
2xx	200 OK, 201 Created, 204 No Content
3xx	301 permanent, 302 Found, **304 Not Modified**, 307/308 keep the method
4xx	400, **401 = not logged in**, **403 = logged in, not allowed**, 404, 429 Too Many Requests
5xx	500, 502 Bad Gateway, 503 Service Unavailable

DNS record	Maps
A / AAAA	name → IPv4 / IPv6
CNAME	name → another name (alias)
MX / TXT / NS	mail servers / arbitrary text (SPF, ownership) / authoritative name servers

Script tag	Download	Runs	Order kept
`<script src>`	blocks parsing	immediately	yes
`defer`	parallel	after parsing, before `DOMContentLoaded`	yes
`async`	parallel	as soon as downloaded	**no**
`type="module"`	parallel	deferred by default	yes

Change	Layout	Paint	Composite
`width`, `margin`, `top`, font size, content	yes	yes	yes
`color`, `background-color`, `box-shadow`	no	yes	yes
`transform`, `opacity`	no	no	yes

Hiding	Takes space	Clickable	Screen readers
`display: none` / `hidden` attr	No	No	No
`visibility: hidden`	Yes	No	No
`opacity: 0`	Yes	**Yes**	**Yes**

In the interview

Q: What happens when you type a URL and press Enter? **A:** URL-or-search decision → DNS (caches, then resolver) → TCP + TLS (or QUIC for HTTP/3) → GET with Host/cookies → response (maybe redirect, ideally 200 HTML) → parse HTML into DOM while fetching discovered CSS/JS/images → CSSOM → render tree → layout → paint → composite; scripts then make it interactive.

Q: How do you make that faster? **A:** Fewer round trips (HTTP/2/3, CDN, keep-alive, `preconnect`), smaller cacheable assets, no unnecessary render-blocking CSS/JS.

Q: Where can caching short-circuit it? **A:** DNS caches, the HTTP cache (no request under `max-age`), a service worker, a CDN edge.

Q: `display:none` vs `visibility:hidden`? **A:** The first removes the box and frees its space; the second keeps the space but draws nothing.

Q: `defer` vs `async`? **A:** Both download in parallel; `defer` runs in order after parsing, `async` runs whenever it arrives, in any order.

Q: Why animate `transform` / `opacity`? **A:** They skip layout and paint — composite only.

Q: 401 vs 403? **A:** 401 = who are you? 403 = I know who you are, and no.

Gotchas

- `<a href="/delete?id=5">` changing data → never mutate with GET; crawlers and prefetchers follow GETs.
- Opening files from disk with root-relative `/css/…` → `/` means the disk root; use a local server or relative paths.
- Secrets in query strings → URLs land in logs, history and `Referer`.
- `CSS/Style.css` vs `css/style.css` → works on a case-insensitive laptop, breaks on a Linux server.
- The Elements panel is the live DOM, not your source (View Source = Ctrl/Cmd+U).
- Missing `<!DOCTYPE html>` / `<meta charset="utf-8">` → quirks mode and garbled characters.

Must-know — networking fundamentals (§1.10)

- **Layers** — TCP/IP model: application (HTTP, DNS, SSH) → transport (TCP, UDP, QUIC) → internet (IP) → link (Ethernet, Wi-Fi). OSI's 7 layers are the teaching version; "L4" = addresses + ports, "L7" = HTTP.
- **Port / socket** — the IP finds the machine, the port finds the program; clients use an ephemeral port. 22 SSH, 53 DNS, 80/443 HTTP(S), 5432 Postgres, 6379 Redis.
- **NAT** — the router rewrites private `IP:port` to its public `IP:port` and keeps a table; unsolicited inbound packets are dropped. Private IPv4: `10/8`, `172.16/12`, `192.168/16`.
- **IPv4 vs IPv6** — 32-bit (exhausted) vs 128-bit (`2001:db8::10`); dual stack + Happy Eyeballs today.
- **Anycast** — one address announced from many places; routing picks the nearest (DNS resolvers, CDNs).
- **More DNS records** — `SOA` (zone authority), `PTR` (reverse), `SRV` (host + port for a service), `CAA` (allowed certificate authorities).
- **Firewall** (packet filter / stateful / WAF at L7); **forward proxy** acts for clients, **reverse proxy** for servers; **VPN** = encrypted tunnel into another network (IPsec, WireGuard, OpenVPN).

	TCP	UDP	QUIC
Set-up	handshake (+TLS)	none	1 RTT incl. TLS 1.3
Delivery	reliable, ordered stream	best effort datagrams	reliable per stream
HOL blocking	yes	n/a	per stream only
Used by	HTTP/1.1, /2, DBs, SSH	DNS, games, calls	HTTP/3

Q: TCP vs UDP? **A:** TCP = handshake + reliable ordered byte stream with congestion control; UDP = independent datagrams with no guarantees and no set-up — for DNS, games and calls where late data is useless.

Q: Why is HTTP/3 on UDP? **A:** Middleboxes already pass UDP; QUIC adds reliability, TLS 1.3 and independent streams itself, and survives network changes.

Q: Forward vs reverse proxy? **A:** Forward acts for clients (egress filtering, caching); reverse acts for servers (TLS termination, load balancing, caching).

TL;DR

URL = scheme+host+port+path+query+fragment; DNS → TCP/TLS or QUIC → HTTP → DOM + CSSOM → render tree → layout → paint → composite. CSS blocks rendering, classic scripts block parsing; `transform` / `opacity` are cheapest.

2. HTML Structure and Text

Elements and attributes, the doctype, the `<head>`, headings, text semantics, lists, links, character references, validation.

Must-know

- **Void elements** — no content, no end tag: `area, base, br, col, embed, hr, img, input, link, meta, source, track, wbr`. `<br />` slash is allowed and ignored.
- **Boolean attributes** — on when present: `disabled="false"` and `required="no"` still mean on.
- `<!DOCTYPE html>` — must be first; switches to **standards mode** (without it: **quirks mode**). Not an element; case-insensitive.
- `<html lang>` — pronunciation, translation, hyphenation; WCAG 3.1.1 Language of Page.
- `<meta charset="utf-8">` — within the **first 1024 bytes**; an HTTP `Content-Type; charset=` header overrides it.
- **Viewport meta** — `width=device-width, initial-scale=1`; without it phones pretend to be ~980px wide. Never `maximum-scale=1` / `user-scalable=no` (fails WCAG 1.4.4).
- `<title>` — required, unique per page, most specific part first; first thing a screen reader announces.
- **Meta description** — snippet candidate, cut at ~150–160 chars; not a ranking trick.
- `rel="canonical"` — preferred URL for duplicate content. `<meta name="robots" content="noindex">` keeps a page out of search.
- **Headings** — one `h1`, don't skip levels going down (going up is fine), pick level by structure not size; `<hgroup>` for heading + tagline. **No document outline algorithm** — removed from the spec in 2022; `<section>` does not reset levels.
- **Global attributes** — `id` (unique), `class`, `lang`, `dir`, `hidden`, `tabindex`, `data-*` (→ `el.dataset`), `inert` (subtree unfocusable/unclickable), `title` (advisory — never essential info).
- **Link vs button** — a link goes somewhere (Enter); a button does something here (Enter or Space).
- `rel` — `noopener` (no `window.opener`; implied by `target="_blank"` in modern browsers), `noreferrer` (also no `Referer`; implies noopener), `nofollow` / `sponsored` / `ugc` (search hints).
- **Escaping** — `<` → `<`, `&` → `&`, matching quotes in attributes → `"` / `'`; `textContent` is safe, raw `innerHTML` is an XSS risk. Comments are public.

Key facts & numbers

Pair	Meaning
`strong` vs `b`	Importance / urgency vs attention without importance
`em` vs `i`	Stress that changes meaning vs different voice (terms, foreign phrases)
`s` vs `del`	No longer accurate (old price) vs removed in an edit
`mark`	Relevant to current context (search match)
`cite`	Title of a work — **not** a person's name
`time datetime` / `data value`	Machine-readable date / other value
`q` vs `blockquote`	Short inline quote (browser adds marks) vs long quote
`span` / `div`	No meaning — hooks only

List	Use
`ul` / `ol` / `dl`	Order doesn't matter / matters (`start`, `reversed`, `type`, `li value`) / name–value (`dt` + `dd`)
Nesting	Sub-list goes **inside** an `li`

Code to remember

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Trail Running Basics — Mountain Club</title>
  <link rel="stylesheet" href="css/style.css">
  <script src="js/app.js" defer></script>
</head>
<body>…</body>
</html>
```

In the interview

Q: What does the doctype do? **A:** Puts the page in standards mode; without it, quirks mode emulates old browser bugs.

Q: `strong` vs `b`, `em` vs `i`? **A:** `strong` / `em` carry importance/stress; `b` / `i` mark attention or a different voice without extra importance.

Q: Why was `rel="noopener"` needed with `target="_blank"`? **A:** The opened page could redirect your tab via `window.opener` (reverse tabnabbing); browsers now imply it.

Q: Link or button for "Add to cart"? **A:** Button — it performs an action on the current page.

Q: Why validate if browsers repair errors? **A:** The repaired DOM may differ from what you meant, and AT, crawlers and selectors all see the repaired tree. Use the Nu checker / html-validate, plus axe or Lighthouse.

Q: Three common validation errors? **A:** Duplicate ids, block elements inside `<p>`, missing `alt`; also bad nesting and non-`li` children in lists.

Q: Can each `<section>` have its own `h1`? **A:** No — the outline algorithm was never implemented and was removed in 2022; levels are what the numbers say.

Gotchas

- `class=big red` unquoted → class `big` plus a stray attribute `red`.
- Heading level chosen for size → style with CSS; choose by structure.
- `href="www.example.com"` → a relative path; include `https://`.
- `<a href="#" onclick>` for actions → use `<button type="button">`.
- `list-style: none` → Safari/VoiceOver stops announcing a list; add `role="list"` if the count matters.
- `download` on a cross-origin URL → ignored.
- Icon-only links → no accessible name.

TL;DR

Doctype → `<html lang>` → head (charset first, viewport, unique title) → body. One `h1`, no skipped levels, elements by meaning, `rel` for security and SEO, escape `<` and `&`, validate.

3. Semantic HTML and Accessibility

Landmarks, headings, the accessibility tree, alt text, ARIA, keyboard and focus, accessible forms, WCAG 2.2, testing.

Must-know

- **Semantic HTML is the foundation** — native elements bring role, name, state, focus and keyboard behaviour for free.
- **Accessibility tree** — each node has **role, name, description, state/properties, value**, exposed to AT via OS APIs.

- **Accessible name order** — `aria-labelledby` → `aria-label` → native label / `alt` → content → `title` / placeholder.
- **First rule of ARIA** — use native HTML first. ARIA changes **semantics only**: no focus, keyboard or behaviour. "No ARIA is better than bad ARIA." Other rules: don't change native semantics; interactive ARIA controls must work by keyboard; never `aria-hidden` / `role="none"` on focusable things; every interactive element needs a name.
- **Alt decision** — decorative/duplicated → `alt=""`; in a link/button → describe destination/action; complex → short alt + full info in text/ `figcaption`; image of text → repeat the text; else describe what matters in context.
- **Live regions** — must exist **before** the text changes; `role="status"` = polite, `role="alert"` = assertive (sparingly).
- `tabindex` — `0` adds a custom widget to Tab order; `-1` = focusable by script only (skip-link targets, dialog containers); **positive values never**.
- `:focus-visible` — matches when the browser thinks an indicator is needed (typically keyboard); never `outline: none` without replacement.
- **Dialogs** — move focus in, trap Tab, Escape closes, return focus to the opener. Native `<dialog>` + `showModal()` makes the rest inert and handles Escape.
- **Accessible forms** — `<label for>` on every field (also enlarges click target), `fieldset` / `legend` for groups, hints/errors via `aria-describedby`, `aria-invalid="true"`, native `required`, `autocomplete` tokens; **placeholders are not labels**; on failed submit move focus to the summary or first invalid field.
- **WCAG 2.2** — POUR (Perceivable, Operable, Understandable, Robust), 13 guidelines, levels A/AA/AAA; **AA** is the usual target; nine new criteria; **4.1.1 Parsing removed**.

Key facts & numbers

Element	Landmark role
`<header>`	`banner` — only when not inside article/aside/main/nav/section
`<nav>`	`navigation` (label multiples: `aria-label="Main"`)
`<search>`	`search` — Baseline 2023, widely available since April 2026
`<main>`	`main` — **one visible per page**, skip-link target
`<aside>`	`complementary`
`<footer>`	`contentinfo` — only when top-level
`<section>` / `<form>`	`region` / `form` — **only if named**
`<article>`	`article` (structure role, not landmark) — self-contained

Contrast	Normal text	Large text (≥24px or ≥18.66px bold)	UI & graphics
AA	**4.5:1**	**3:1**	**3:1** (1.4.11)
AAA	7:1	4.5:1	—

Contrast ratio = `(L_lighter + 0.05) / (L_darker + 0.05)`, range 1:1 to 21:1; luminance `L = 0.2126 R + 0.7152 G + 0.0722 B` (after linearising sRGB).

Key SC	Level	Rule
1.4.4 Resize Text	AA	Works at 200% text size
1.4.10 Reflow	AA	320 CSS px wide (400% zoom), no horizontal scroll
2.4.7 Focus Visible	AA	Visible focus indicator
4.1.2 Name, Role, Value	A	Correct name, role, state
4.1.3 Status Messages	AA	Announced without moving focus
New 2.2: 2.4.11 Focus Not Obscured (Min)	AA	Focus not entirely hidden by sticky headers/banners
2.5.7 Dragging Movements	AA	Single-pointer alternative to drag
2.5.8 Target Size (Min)	AA	≥ **24×24 CSS px** (or spacing)
3.2.6 Consistent Help	A	Help in the same place
3.3.7 Redundant Entry	A	Don't ask twice
3.3.8 Accessible Authentication (Min)	AA	No cognitive tests; allow paste and password managers

Screen reader	Platform
NVDA (free) / JAWS	Windows — `H` heading, `D` landmark
VoiceOver	macOS (Cmd+F5; VO+U rotor), iOS
TalkBack	Android

Code to remember

```
<button type="button" aria-expanded="false" aria-controls="menu">Menu</button>
<label for="email">Email</label>
<input id="email" type="email" autocomplete="email" required
       aria-describedby="email-err" aria-invalid="true">
<p id="email-err">Error: enter an address like name@example.com</p>
<div role="status"></div>  <!-- exists before content is injected -->
```

In the interview

Q: What is the accessibility tree? **A:** A tree the browser derives from the DOM, with role, name, description, states and value per node, exposed to screen readers through OS APIs.

Q: What is the first rule of ARIA? **A:** Use native HTML when it exists; ARIA only changes semantics and adds no behaviour, so a `<button>` beats `<div role="button">`.

Q: `aria-label` vs `aria-labelledby`? **A:** A string on the element vs a reference to visible text elsewhere; `labelledby` wins.

Q: What does `aria-hidden="true"` do? **A:** Removes the subtree from the accessibility tree; it doesn't hide visually or stop focus.

Q: Why not `tabindex="5"`? **A:** It jumps ahead of all natural focus order, so order no longer matches the page.

Q: Can a `<div onclick>` be a button to a screen reader? **A:** No — no role, no button name, not focusable.

Q: Does CSS affect the accessibility tree? **A:** Yes — `display:none` / `visibility:hidden` remove nodes; `::before` text can join names; `opacity:0` hides nothing.

Q: How do you test accessibility? **A:** axe/Lighthouse in dev and CI, then a manual keyboard pass, then a screen-reader pass on key flows; automation finds only part of the issues.

Q: Can Lighthouse score 100 and still be inaccessible? **A:** Yes; it checks a subset of rules and nothing about meaning, order or usability.

Gotchas

- Div soup / several `<main>` / unlabelled multiple `<nav>`s.
- `<section>` everywhere "because HTML5" → no heading, no value.
- Colour alone for errors → add text ("Error:") and a thicker border (1.4.1).
- Hover-only menus → must open on focus too.
- Adding a visual `*` for required → `aria-hidden` it; native `required` already announces it.
- ARIA on a div widget → pages with ARIA average more detected errors (WebAIM).

TL;DR

Native elements first; role + name + state per control; `alt` by purpose; `tabindex` 0/-1 only, visible `:focus-visible` ; labels, not placeholders; WCAG 2.2 AA = 4.5:1 text, 3:1 large text and UI, 24×24 targets; test automated + keyboard + screen reader.

4. Images, Media and Embeds

`<img>` and reserved space, formats, responsive images, loading hints, figures, audio/video, iframes, SVG, canvas.

Must-know

- **`<img src alt width height>`** — every time. `width` / `height` are unitless integers giving the **aspect ratio**, so space is reserved (no layout shift). Fluid: `max-width: 100%; height: auto` .
- **`alt=""` vs no `alt`** — empty = decorative, skip it; missing = error (AT may read the URL).
- **`srcset` `x` descriptors** — fixed-size image, sharp on high-DPR screens.
- **`srcset` `w` descriptors + `sizes`** — fluid images; `sizes` tells the browser the display width **before CSS loads**; the browser divides file width by slot width to get density, then picks for the DPR. Without `sizes` it assumes `100vw` .
- **`<picture>`** — you choose: art direction (`<source media>`) or format fallback (`<source type>`); first matching source wins; the `<img>` child is mandatory and carries `alt` and dimensions.
- **`loading="lazy"`** — below the fold and iframes; **never on the hero / LCP image**. **`fetchpriority="high"`** — on the single most important image (Baseline since Oct 2024).
- **`figure` / `figcaption`** — referenced, movable content; caption adds context, alt replaces the image — don't duplicate.
- **Media** — `controls` , ordered `<source type>`s, `<track kind="captions">` (WebVTT). **Autoplay only when `muted` ;** `playsinline` for iOS. Big video → adaptive streaming (HLS/DASH).
- **Captions vs subtitles** — captions include non-speech sounds; subtitles translate dialogue.
- **`<iframe>`** — needs `title` ; `loading="lazy"` ; `sandbox` with fewest tokens; `allow` for permissions. Empty `sandbox` blocks scripts, forms, pop-ups, top navigation, plugins and gives an opaque origin. Never `allow-scripts allow-same-origin` on untrusted same-origin content.
- **Clickjacking defence** — `Content-Security-Policy: frame-ancestors` .
- **SVG** — always set `viewBox` ; `symbol` + `use` + `currentColor` for icons; never inline untrusted SVG (scripts run).
- **Canvas** — script-painted bitmap; set buffer size by attributes × `devicePixelRatio` (CSS-only sizing blurs); no built-in accessibility.

Key facts & numbers

Format	Type	Transparency	Animation	Best for
JPEG	lossy raster	no	no	photos, universal fallback
PNG	lossless raster	yes	no	screenshots, UI, sharp text
GIF	256 colours	1-bit	yes	legacy — prefer video/WebP/AVIF
WebP	lossy/lossless	yes	yes	JPEG/PNG replacement (Safari since 2020)
AVIF	lossy/lossless	yes	yes	smallest photos; **Baseline 2024, widely available since July 2026**
SVG	vector (XML)	yes	yes	icons, logos, illustrations

Need	Use
Fixed size, high-DPR	`srcset` with `1x, 2x`
Fluid, many screens	`srcset` `w` + `sizes`
Different crop per layout	`<picture>` + `<source media>`
Modern format + fallback	`<picture>` + `<source type>`

	Inline `<svg>`	`<img src=svg>`
Style with page CSS / `currentColor`	yes	no
Cached separately	no	yes
Scripts run	yes	no

	SVG	Canvas
Model	retained (DOM nodes)	immediate (pixels)
Scales without blur / per-shape events / a11y	yes	no
Sweet spot	up to a few thousand elements	many thousands, games, real-time

Code to remember

```
<img src="hero-800.jpg" alt="Runners on a ridge at sunrise" width="800" height="533"
     srcset="hero-400.jpg 400w, hero-800.jpg 800w, hero-1600.jpg 1600w"
     sizes="(min-width: 60rem) 50vw, 100vw" fetchpriority="high">
<picture>
  <source srcset="a.avif" type="image/avif">
  <source srcset="a.webp" type="image/webp">
  <img src="a.jpg" alt="…" width="800" height="600" loading="lazy">
</picture>
```

In the interview

Q: How do you make images responsive? **A:** `max-width:100%; height:auto`; `srcset` (`x` or `w` + `sizes`); `<picture>` for art direction/format; `width` / `height` against layout shift.

Q: Why can't the browser just measure the layout? **A:** Image downloads start during HTML parsing, before CSS and layout exist.

Q: How do you improve image loading? **A:** AVIF/WebP, right size, compress, lazy below the fold, `fetchpriority="high"` on the LCP image, dimensions, a CDN.

Q: Which file did the browser choose? **A:** `img.currentSrc` or the Network panel.

Q: Why won't my autoplay video start? **A:** Audible autoplay is blocked; add `muted` (and `playsinline` on iOS).

Q: How to embed untrusted HTML? **A:** Sandboxed iframe (no `allow-same-origin`), maybe `srcdoc`, or sanitise.

Q: Why do payment providers use iframes? **A:** The card field lives on their origin, so the merchant page's scripts can't read it; reduces PCI scope.

Q: Canvas or SVG for a chart? **A:** SVG for typical dashboards (accessible, per-element events); canvas for tens of thousands of points or real-time.

Gotchas

- `width="800px"` → attributes take plain integers.
- `w` descriptors without `sizes`, or mixing `x` and `w` → too-big downloads / invalid.
- `alt` on `<source>` → belongs on the `<img>`.
- `fetchpriority="high"` everywhere → nothing is high.
- Video fallback text shows for unsupported *format* → no, only for browsers without `<video>`.
- Missing `viewBox` → SVG crops instead of scaling.

TL;DR

Dimensions + purposeful alt on every image; AVIF/WebP with fallback; `srcset` + `sizes` for the browser to choose, `<picture>` for you to choose; lazy below the fold, never the hero; iframes titled and sandboxed.

5. Tables and Forms

Table semantics, how forms submit, input types, grouping controls, built-in validation, `autocomplete`, accessible errors.

Must-know

- **Tables are for data only** — `<caption>` (the accessible name, first child), `thead` / `tbody` / `tfoot`, `<th scope="col|row|colgroup|rowgroup">`; `headers="id1 id2"` on `<td>` for irregular tables. Wide tables → scrollable, focusable wrapper.
- **`border-collapse`** — `collapse` merges adjacent borders; `separate` (default) draws each with `border-spacing`.
- **Successful controls — named, enabled, and (checkbox/radio) checked** are sent to `action` using `method`.
- **GET vs POST** — GET puts data in the URL (searches, filters); POST in the body (changes, private data); files need `enctype="multipart/form-data"`.
- **Buttons** — default `type="submit"`; use `type="button"` for script actions; `reset` is rarely wanted.
- **Disabled vs readonly** — disabled: not focusable, not submitted, not validated. Readonly: focusable, submitted, not editable, not validated.
- **JS submit** — listen for `submit`, `preventDefault()`, `fetch` with `new FormData(form)`. `form.requestSubmit()` runs validation and fires `submit`; `form.submit()` skips both.
- **Input types** — pick the most specific (keyboard + validation). `tel` has **no** validation; `number` only for real quantities; `month` / `week` unsupported in desktop Firefox and Safari; `hidden` is trivially editable. `inputmode` changes the keyboard without validation side effects.
- **Grouping** — `fieldset` + `legend` for every radio/checkbox group (legend announced with each control); `disabled` on fieldset disables all inside. Radios share one `name`.
- **`datalist`** = suggestions, not restriction. **`textarea`** initial value is its content. **`output`** is a live region, not submitted. **`progress`** = task completion; **`meter`** = measurement in a range (`low` / `high` / `optimum`).
- **Constraint Validation API** — `validity` flags, `validationMessage`, `checkValidity()` (boolean + `invalid` event), `reportValidity()` (also shows the message), `setCustomValidity(msg)` (**reset with `''`**).
- **`:user-invalid`** — matches only after interaction or submit attempt; Baseline 2023, widely available since May 2026. `:invalid` makes forms load red.

- **`autocomplete`** — `current-password` vs `new-password` (triggers strong-password suggestions), `one-time-code`, `street-address`, `cc-number` …; don't disable autofill or paste (WCAG 3.3.8).
- **Server validation is mandatory** — client validation is UX only.

Key facts & numbers

Attribute	`validity` flag
`required`	`valueMissing`
`type=email/url`	`typeMismatch`
`pattern` (implicitly anchored `^…$`)	`patternMismatch`
`minlength` / `maxlength` (only after user edit)	`tooShort` / `tooLong`
`min` / `max`	`rangeUnderflow` / `rangeOverflow`
`step` (base = `min`, else initial value, else 0)	`stepMismatch`
unparsable input	`badInput`
`setCustomValidity("msg")`	`customError`

Not submitted
Unnamed, disabled, unchecked controls; unselected options; buttons other than the one used; `<output>`

Code to remember

```
form.addEventListener('submit', async (e) => {
  e.preventDefault();
  if (!form.checkValidity()) return showErrors();   // works with novalidate
  await fetch(form.action, { method: 'POST', body: new FormData(form) });
});
confirm.addEventListener('input', () =>
  confirm.setCustomValidity(confirm.value === pw.value ? '' : 'Passwords do not match.'));
```

In the interview

Q: How do you make a data table accessible? **A:** `caption`, `th` with `scope`, `thead/tbody/tfoot`, `headers` for irregular tables, a scroll container on small screens.

Q: What does `scope` do? **A:** Tells AT which cells a header describes — its column, row, or spanned group.

Q: Why does my `<button>` refresh the page? **A:** Default `type="submit"`.

Q: Why a label rather than a placeholder? **A:** Accessible name, larger click area, persists while typing.

Q: How does built-in validation work? **A:** Constraint attributes → `validity` flags; `setCustomValidity` for extra rules; style with `:user-invalid`; `novalidate` + custom UI for full control (the API still works).

Q: `checkValidity` vs `reportValidity`? **A:** Both return a boolean and fire `invalid`; `reportValidity` also shows the message.

Q: Accessible error pattern? **A:** `novalidate`; on submit set `aria-invalid`, specific text next to each field via `aria-describedby`, an error summary with links that receives focus; never colour alone; keep input.

Gotchas

- `<td><b>` as headers → use `<th>`. Obsolete `summary` attribute → caption or nearby text.
- Radios with different `name`s → don't exclude each other.
- Forgetting `setCustomValidity('')` → field can never become valid.
- Over-strict `pattern` for names ("O'Brien", "José").
- `autocomplete="off"` on logins → weaker reused passwords; browsers often ignore it.
- Disabling submit until valid → users can't discover why.

- `<progress>` for a static measurement → `<meter>`.

TL;DR

Tables: caption + `th scope`. Forms send named, enabled, checked controls; GET reads, POST changes; label everything; declare constraints, read `validity`, style `:user-invalid`, re-validate on the server.

6. CSS Fundamentals

Adding CSS, selectors, pseudo-classes/elements, specificity, the cascade and origins, inheritance, units, colours, custom properties.

Must-know

- **Rule** — selector + `{ property: value; }`. Invalid declarations are skipped, not fatal. External stylesheets for real sites; inline only when you must (beats selectors, no media queries or pseudo-classes).
- **Selector list** — one invalid selector drops the whole rule, except inside forgiving `:is()` / `:where()`.
- **Combinators** — descendant (space, any depth), child `>`, next sibling `+` (immediately after), subsequent siblings `~`. `+` / `~` only look at **later** siblings with the **same parent**. `.nav .active` (descendant) ≠ `.nav.active` (same element).
- **Attribute selectors** — `[a]`, `[a="v"]`, `[a~="v"]` (word), `[a|="v"]` (v or v-…), `[a^="v"]`, `[a$="v"]`, `[a*="v"]`.
- **Pseudo-class vs pseudo-element** — `:` state/position (`:hover`, `:nth-child`, `:checked`, `:has`) vs `::` part/generated box (`::before` **needs** `content`; not rendered on `<img>` and most inputs).
- **LVHA** — `:link`, `:visited`, `:hover`, `:active` order. `:visited` limited to colour properties (privacy).
- **An+B** — `odd` = `2n+1`, `even` = `2n`, `-n+3` = first three; `:nth-child(2 of .item)` counts only `.item`s. `:nth-child` counts all siblings, `:nth-of-type` only same type.
- **`:has()`** — the "parent selector": matches by contents or following siblings. Baseline since December 2023, widely available since June 2026.
- **Specificity** — see the scoring rule below.
- **Cascade** — see the order below. "The last rule wins" only among equal origin, layer and specificity.
- **`!important`** — legitimate for always-apply utilities (`.hidden{display:none!important}`), overriding third-party inline styles, user stylesheets; otherwise a smell.
- **Inheritance** — text properties inherit (`color`, `font`, `line-height`); box properties don't; any matching rule beats inheritance.
- **Custom properties** — `--name: value` + `var(--name, fallback)`; cascade and inherit; resolved at runtime; editable with `setProperty`; fallback only covers **undefined**; invalid substitution acts like `unset`; not usable in media query conditions. `@property` registers a type and initial value (Baseline 2024) and makes them animatable.

Key facts & numbers

Specificity scoring rule (A, B, C):

Column	Counts
A	ID selectors
B	classes, attribute selectors, pseudo-classes
C	type selectors, pseudo-elements
(zero)	`*`, combinators, `:where(…)`

1. `:is()`, `:not()`, `:has()` add the specificity of their **most specific argument** (even if a less specific one matched).
2. `:where()` always adds **zero**.
3. `:nth-child(An+B of S)` = one pseudo-class **plus** the most specific selector in `S`.

4. Compare A, then B, then C — **no carrying**: `(0,11,0)` loses to `(1,0,0)`.
5. Inline `style=""` isn't scored — it beats any selector; `!important` beats both.
6. Repeating a class (`.btn.btn`) really adds specificity.

Selector	Score
`ul > li + li`	(0,0,3)
`a:hover` , `input[type="email"]`	(0,1,1)
`p::first-line`	(0,0,2)
`#main .card p`	(1,1,1)
`:is(#a, .b, p) span`	(1,0,1)
`:where(#a, .b, p) span`	(0,0,1)
`.card:has(> img)`	(0,1,1)
`li:nth-child(2n+1 of .item)`	(0,2,1)
`.nav > li:first-child a:hover`	(0,3,2)
`#nav ul li a:hover`	(1,1,3)

Cascade order (CSS Cascading 5) — compared in order, stop at first difference:

1. **Origin and importance**
2. **Context** (shadow DOM)
3. **Element-attached styles** (`style=""`)
4. **Cascade layers**
5. **Specificity**
6. **Order of appearance** (later wins)

(With `@scope` rules, **scope proximity** is compared after specificity and before order of appearance — §14.6 Q15, §13.5.)

Rank (strongest first)	Declarations
1	CSS transitions (while running)
2	`!important` user-agent
3	`!important` user
4	`!important` author
5	CSS animations (while running)
6	normal author
7	normal user
8	normal user-agent

Layers: normal declarations — later layers beat earlier, **unlayered beats all layers**. `!important` — flips: earlier layers win, layered `!important` beats unlayered `!important` . `@layer` Baseline since 2022.

Keyword	Meaning
`inherit`	Parent's computed value
`initial`	Spec initial value (`display: initial` = `inline`)
`unset`	`inherit` if inherited property, else `initial`
`revert`	Previous origin (usually browser default: `div` → `block`)
`revert-layer`	Previous cascade layer

Unit	Relative to
`px`	CSS pixel (1/96 in), not a device pixel
`em`	element's font-size — **parent's when used in** `font-size` (compounds)
`rem`	root font-size (16px default)
`%`	width/padding/margin (**incl. top/bottom**) → containing block **width**; height → its height; font-size → parent
`vw` / `vh` , `svh` / `lvh` / `dvh`	viewport; small/large/dynamic heights (Baseline 2022)
`ch`	width of "0" — `max-width: 65ch`

Colour	Note
`rgb(102 51 153 / 50%)`	space syntax + `/ alpha` ; `rgba` is an alias; don't mix commas with `/`
`oklch(L C H)`	perceptually uniform lightness; Baseline since 2023
`color-mix(in oklch, a 50%, b)`	Baseline since 2023
`currentColor`	the element's `color`

Code to remember

```
@layer reset, base, components;              /* later = stronger */
@import url(lib.css) layer(vendor);          /* third-party in a low layer */
:where(ul, ol) { margin: 0; }                 /* (0,0,0) — anything overrides */
:root { --brand: oklch(44% 0.16 303); }
.btn { background: var(--brand, rebeccapurple); }
```

In the interview

Q: Explain the cascade. **A:** Origin & importance → inline style → layers → specificity → source order; `!important` reverses origins and layers.

Q: Does `.a.a` beat `.a`? **A:** Yes, (0,2,0) vs (0,1,0).

Q: `:is()` vs `:where()`? **A:** Same matching; `:is` takes its most specific argument's specificity, `:where` is zero.

Q: Where do you put a third-party library? **A:** In an early `@layer` so your later/unlayered rules win without specificity tricks.

Q: `em` vs `rem`? **A:** `em` follows the element's (or parent's, in font-size) size and compounds; `rem` follows the root and respects user settings.

Q: `initial` vs `unset` vs `revert`? **A:** Spec default / inherit-or-initial / previous origin's value.

Q: Custom properties vs Sass variables? **A:** Runtime, cascading, per-element, JS-accessible (theming); Sass is build-time but usable in selectors/media conditions.

Q: Style a card that contains an image without JS? **A:** `.card:has(img) { … }`.

Gotchas

- `li:first-child` when the first child is a `div` → nothing matches.
- `:focus { outline: none }` → style `:focus-visible` instead.
- `height: 100vh` on mobile → `100dvh` / `svh`. `width: 100vw` → horizontal scrollbar (includes scrollbar) → `100%`.
- `padding-top: 50%` is half the **width** (use `aspect-ratio` now).
- `opacity: 0.5` for a translucent background → fades children too; use an alpha colour.
- `color: --brand` → needs `var()`.

TL;DR

Specificity (IDs, classes/attrs/pseudo-classes, types/pseudo-elements), no carrying, `:where` = 0. Cascade: origin & importance → inline → layer → specificity → order. Text inherits; `rem` for type; tokens on `:root`.

7. The Box Model and Layout Basics

Box model, `box-sizing`, margin collapsing, `display`, normal flow and BFCs, positioning, overflow, stacking contexts, floats.

Must-know

- **Box model** — content → padding → border → margin. Margin is transparent and outside; background covers content + padding (under the border).
- **`box-sizing`** — `content-box` (default): `width` = content, padding + border **added** (200 + 2×20 + 2×6 = **252**). `border-box`: `width` includes padding + border; content shrinks. Margin is never included. **Not inherited** → set on `*, *::before, *::after`.
- **Clock rule** — `margin: T R B L`; 2 values = vertical horizontal; 3 values = T, horizontal, B.
- **`width: auto` ≠ `100%`** — `100%` + padding on content-box overflows. `height: 100%` fails if the parent's height is `auto`.
- **Border vs outline vs box-shadow** — only `border` takes space; outline and shadow don't shift layout.
- **Margin collapsing** — vertical margins in block flow merge: between siblings, parent and first/last child (nothing separating), empty blocks. **Never** horizontal, never in flex/grid, never for floats/absolute, not across padding/border/new BFC.
- **`display`** = outer (block/inline) + inner (flow/flow-root/flex/grid). **Inline** ignores `width`/`height` and vertical margins; vertical padding paints but doesn't move lines. **Inline-block** keeps size, sits in the line. `none` removes the box; `contents` removes only the element's own box (check accessibility).
- **BFC** — created by root, `display: flow-root`, `inline-block`, floats, absolute/fixed, `overflow` not `visible`/`clip`, table cells, flex/grid items, `contain: layout/paint`, multicol. It **contains floats** and stops margin collapse through its edges. Prefer `flow-root` over `overflow: hidden` (which clips shadows and focus rings).
- **Positioning** — `relative` shifts but keeps space (anchor for absolute children); `absolute` out of flow, against nearest positioned ancestor's **padding box** (else initial containing block); `fixed` against the viewport — **unless an ancestor has `transform`/`filter`/`will-change`**; `sticky` in flow until its threshold in the nearest scroll container, never leaves its parent.
- **Overflow** — `hidden` clips but is script-scrollable; `clip` clips, not scrollable, no BFC; `scroll` always bars; `auto` bars when needed. Ellipsis = `white-space: nowrap; overflow: hidden; text-overflow: ellipsis`.
- **Stacking context** — `z-index` works on positioned and flex/grid items, **only within its stacking context**; children are flattened into it. The **top layer** (modal `<dialog>`, popover, fullscreen) beats every z-index.
- **Floats** — only for wrapping text around images (`shape-outside`); contain with `flow-root`, escape with `clear`; never for layout.

Key facts & numbers

Margins meeting	Result
All positive (30, 20)	Largest: **30**
Mixed (30, −10)	Largest positive + most negative: **20**
All negative (−10, −25)	Most negative: **−25**

	block	inline	inline-block
New line	Yes	No	No
`width` / `height`	Respected	**Ignored**	Respected
Vertical margin	Applies (may collapse)	**Ignored**	Applies (never collapses)

Creates a stacking context
Root; `position: relative/absolute` **with** `z-index` ≠ `auto`; `fixed` / `sticky` (always); flex/grid item with `z-index` ≠ `auto`; `opacity` < 1; `transform`, `scale`, `rotate`, `translate`, `filter`, `backdrop-filter`, `perspective`, `clip-path`, `mask`; `mix-blend-mode` ≠ `normal`; `isolation: isolate`; `will-change` of those; `contain: layout/paint/strict/content`; `container-type: size/inline-size`

Measure API	Returns
`getBoundingClientRect()`	Border box on screen, incl. transforms, fractional
`offsetWidth`	Border box, rounded, ignores transforms
`clientWidth`	Padding box minus scrollbar

Centring	Technique
Inline content	`text-align: center`
Block with max width	`max-width: 40rem; margin-inline: auto`
Block children vertically	`align-content: center` on a plain block (2024, Baseline newly available)
Both ways	`display: grid; place-items: center`

Code to remember

```
*, *::before, *::after { box-sizing: border-box; }
.card    { display: flow-root; }          /* contain floats, stop margin leak */
.overlay { position: absolute; inset: 0; margin: auto; }
.header  { position: sticky; top: 0; }
.wrap    { isolation: isolate; }          /* new stacking context, nothing else */
```

In the interview

Q: Explain the box model and `box-sizing`. **A:** Content, padding, border, margin; content-box adds padding and border to `width` (200 → 252), border-box includes them; margin is never included.

Q: Why did my `h2`'s margin push the whole section down? **A:** Parent–child collapse; add padding or `display: flow-root` to the section.

Q: What is a BFC? **A:** An independent block layout region (e.g. `flow-root`) that contains floats and stops margin collapse through its edges.

Q: Why doesn't `z-index: 9999` work? **A:** It only competes inside its stacking context; if an ancestor's context sits below a sibling's, nothing inside can escape. Compare the ancestors that are siblings in a common context.

Q: `z-index: auto` vs `0`? **A:** Same paint level, but `0` creates a stacking context.

Q: Why doesn't my sticky header stick? **A:** No `top`, an ancestor with `overflow` that isn't the scroller, or the parent is too short.

Q: Why does my `fixed` modal scroll with the page? **A:** An ancestor has `transform` / `filter` / `will-change`; move it out or use `<dialog>`.

Q: Do horizontal margins collapse? **A:** Never.

Gotchas

- `border: 1px red` → no style, nothing shows.
- `width` on `<a>` / `<span>` → no effect; make it inline-block/block.
- `inline-block` card rows → whitespace gaps; use flex/grid with `gap`.
- Absolute with no positioned ancestor → jumps to the page corner.
- `top:50%; left:50%` without `translate: -50% -50%`.
- `overflow: hidden` on `body` → breaks sticky below it.
- `display: none` for screen-reader text → use a visually-hidden class.

TL;DR

border-box everywhere; vertical margins collapse only in block flow; `flow-root` for BFCs; absolute → nearest positioned ancestor, fixed → viewport unless transformed ancestor, sticky → scroll container within parent; `z-index` is local to its stacking context.

8. Flexbox

One-dimensional layout: container and items, two axes, grow/shrink/basis, the `flex` shorthand, automatic minimum sizes, alignment, wrapping, patterns.

Must-know

- **Flex container** — `display: flex` makes **direct children** flex items: blockified, no margin collapsing, floats ignored.
- **Container props** — `flex-direction`, `flex-wrap` (`flex-flow`), `justify-content`, `align-items`, `align-content`, `gap`. **Item props** — `flex-grow/shrink/basis` (`flex`), `align-self`, `order`, `margin` (incl. `auto`), `min-width`.
- **Axes** — main axis = `flex-direction` (`row` default, follows text direction); cross axis perpendicular. `justify-content` → main; `align-items` / `align-self` / `align-content` → cross. Swap direction and meanings swap.
- **`order`** / **`*-reverse`** — visual only; Tab and reading order unchanged (`reading-flow` is Chromium-only, not Baseline) → change the HTML instead.
- **`flex-basis`** — starting size; `auto` = use `width` / `height` if set, else content. Default `auto`.
- **`flex-grow`** — shares of **positive free space** (default 0). It divides free space, **not final widths**; ratios equal width ratios only when bases are 0.
- **`flex-shrink`** — gives up space on overflow **weighted by basis** (default 1), so small items aren't crushed first.
- **`min-width: auto`** — for flex items means "never below min-content" → the classic overflow bug; fix `min-width: 0` (a scroll container also removes it; `overflow: clip` does not).
- **Auto margins** absorb free space: `margin-left: auto` pushes right, `margin-top: auto` pushes down, `margin: auto` centres (and never overflows past the start edge).
- **`stretch`** (default `align-items`) works only when the item's cross size is `auto`.
- **`safe center`** — falls back to start when overflowing (Baseline since 2024).
- **Wrap** — each line sized independently; `align-content` only matters with wrapping; an even last row of cards → use grid.

Key facts & numbers

Flex sizing algorithm (Flexbox L1 §9.7, one line):

1. Base sizes from `flex-basis`, clamped by min/max → hypothetical sizes.

2. Sum hypothetical sizes + gaps, margins, padding, borders. Less than container → grow; else shrink.
3. Freeze items whose factor is 0.
4. Distribute to unfrozen items:
    - **grow:** `size = basis + free × grow / Σgrow`
    - **shrink:** `size = basis − overflow × (shrink × basis) / Σ(shrink × basis)`
    - if unfrozen factors sum to **< 1**, only that fraction of free space is handed out.
5. Clamp to min/max; freeze clamped items; repeat step 4 with remaining space.
6. Stop when all frozen.

Worked (verified in Chrome)	Result
Grow: 600px, `1 1 100px` / `2 1 150px` / `1 1 50px` → free 300, share 75	**175 / 300 / 125**
Shrink: 400px, `0 1 300px` / `0 1 200px` / `0 2 100px` → overflow 200, weights 300/200/200 (Σ700)	**214.29 / 142.86 / 42.86**
Max-width freeze	100 / 250 / 250
`flex:1` vs `flex:auto` (G5)	200/200/200 vs 160/253/187

Shorthand	= grow shrink basis	Use
default / `initial`	`0 1 auto`	content-sized, may shrink
`flex: 1`	`1 1 0%`	share **all** space equally
`flex: 2`	`2 1 0%`	twice a `flex:1` sibling
`flex: auto`	`1 1 auto`	content size + share **leftover**
`flex: none`	`0 0 auto`	rigid (icons, avatars)
`flex: 200px`	`1 1 200px`	wrapping card rows
`flex: 0 0 200px`	fixed	sidebars

justify-content	Free space goes
`flex-start`	at the end
`center`	half each side
`space-between`	between items only
`space-around`	around each item (edges get half)
`space-evenly`	equal everywhere incl. edges

Code to remember

```
.center  { display: flex; justify-content: center; align-items: center; }
.nav     { display: flex; gap: 1rem; } .nav .signin { margin-left: auto; }
.page    { display: flex; flex-direction: column; min-height: 100dvh; }
.page > main { flex: 1; }                        /* sticky footer */
.media   { display: flex; gap: 1rem; } .media img { flex: none; }
.media .body { min-width: 0; }                   /* allow truncation */
```

In the interview

Q: Explain flexbox in 30 seconds. **A:** One-dimensional layout; direct children placed along a main axis; `justify-content` for main, `align-*` for cross; sizes from basis, then grow shares positive space and shrink takes back overflow weighted by shrink × basis, clamped by min/max.

Q: `flex: 1` vs `flex: auto`? **A:** Basis 0% vs auto: `flex: 1` ignores content and splits everything; `flex: auto` keeps content differences and splits only the leftover.

Q: How is shrinking calculated? **A:** overflow × (shrink × basis) / Σ(shrink × basis) — weighted by size.

Q: Why does my `flex: 1` item overflow? **A:** `min-width: auto` = min-content; set `min-width: 0` and handle the content (ellipsis or `overflow-wrap: anywhere`).

Q: How do you centre a div? **A:** Flex with `justify-content` + `align-items: center`, or `display: grid; place-items: center`.

Q: Flexbox or grid for page layout? **A:** Grid for two-dimensional structure and uniform card grids; flexbox for content-sized rows inside components; often nested.

Q: Why does `justify-content` do nothing? **A:** An item grows, so no free space remains.

Gotchas

- `flex-grow: 2` ≠ twice as wide unless bases are equal (typically 0); padding counts toward the base.
- Everything `flex-shrink: 0` → row overflows on phones.
- Images in a flex row get squashed (`flex: none`) or stretched (`align-self: flex-start`).
- `align-items: center` to centre horizontally in a row → that's `justify-content`.
- Explicit `height` on items kills `stretch`.
- `justify-content: center` on an overflowing row → left part unreachable; use `safe center` or auto margins.
- `<input>` in a flex row refuses to shrink → `min-width: 0`.

TL;DR

Basis first, then grow shares free space (`basis + free × grow/Σgrow`), shrink takes overflow by `shrink × basis`; clamp, freeze, repeat. `flex: 1` = `1 1 0%`. Fix overflow with `min-width: 0`. Auto margins push things apart.

9. CSS Grid

Two-dimensional layout: lines, tracks, areas, `fr` and `minmax()`, `auto-fill` vs `auto-fit`, placement, the implicit grid, alignment, subgrid, grid vs flexbox.

Must-know

- **Grid** = two-dimensional, **layout-in** (container's tracks decide sizes); flexbox is one-dimensional, **content-out**.
- **Vocabulary** — **lines** numbered from 1 (and −1 backwards; 3 columns = 4 lines), **tracks** (rows/columns), **cells**, **areas**, **gaps** (only between tracks, never at edges). **Explicit** grid = declared; **implicit** grid = extra tracks the browser creates.
- `fr` — a fraction of the **leftover space after fixed tracks and gaps**. `%` tracks ignore gaps, so `50% 50%` + gap overflows; `1fr 1fr` never does.
- **`1fr` = `minmax(auto, 1fr)`** — the `auto` minimum (min-content) makes columns unequal with long content; use **`minmax(0, 1fr)`** for truly equal columns (grid twin of `min-width: auto`).
- **Track sizing keywords** — `auto` (content as max, stretches when no `fr` tracks; automatic minimum as min), `min-content` (longest word), `max-content` (no wrapping), `fit-content(150px)` (max-content capped at 150px, at least min-content), `minmax(min, max)`.
- **RAM pattern** — `repeat(auto-fill|auto-fit, minmax(min(X, 100%), 1fr))`: responsive with no media queries; `min(X, 100%)` stops overflow on narrow screens.
- **auto-fill vs auto-fit** — differ only with fewer items than tracks: **auto-fill keeps empty tracks**, **auto-fit collapses them** (and their gaps) so items stretch.
- **Placement** — `grid-column: 1 / 3`, `span 2`, `1 / -1` (full width); `grid-area: row-start / col-start / row-end / col-end` (anticlockwise, unlike margin); named lines; `grid-template-areas` (rectangles only, `.` = empty; mismatched strings invalidate the whole declaration). Items can overlap in one cell (`grid-area: 1 / 1`).

- **Implicit grid** — `grid-auto-rows/columns` size extra tracks; `grid-auto-flow: row | column | dense` (`dense` back-fills holes and breaks DOM/visual order).
- **Alignment** — `justify-*` = inline axis, `align-*` = block axis. `*-items` / `*-self` inside cells (default `stretch`, but items with an aspect ratio like images → `start`); `*-content` moves the whole grid only when tracks leave free space (no effect with `fr`). `place-*` = align then justify.
- **Subgrid** — `grid-template-rows: subgrid` on a spanning item reuses the parent's tracks so card titles/text/buttons align. Baseline since Sept 2023.
- **Masonry** (`display: grid-lanes`) is Safari-only in 2026 — not Baseline.

Key facts & numbers

Grid track sizing essentials:

- `fr` **calculation:** container inner size − fixed tracks − gaps = free space; ÷ Σfr. `200px 1fr 2fr`, gap 20, 700px → 700 − 200 − 2×20 = **460** → **1fr = 153.33**, 2fr = 306.67. A track whose share is below its minimum is fixed at that minimum and the rest re-split.
- **Auto-repeat count: n = floor((W + gap) / (min + gap))**; then `1fr` stretches the n tracks. 700px, gap 10, min 120 → n = floor(710/130) = 5, track = (700 − 40)/5 = **132**. auto-fit with 3 items → (700 − 20)/3 = **226.67**.
- **Unequal** `1fr 1fr` with a long URL: 449 / 243 → `minmax(0,1fr)` gives 346 / 346.

	Flexbox	Grid
Dimensions	one	two
Sizes decided by	content (content-out)	container tracks (layout-in)
Rows align in columns	no	yes
Wrapped last row	lonely item stretches	keeps column width
Overlap	needs positioning	same cell
Cross-sibling alignment	impossible	`subgrid`
Typical	navbars, toolbars, tags, media objects	page layouts, galleries, dashboards, forms

Pattern	Recipe
Centre anything	`display: grid; place-items: center`
Overlay caption	image and caption both `grid-area: 1 / 1`
Full-bleed	`[full-start] 1fr [content-start] minmax(0, 60rem) [content-end] 1fr [full-end]`
Aligned form	`grid-template-columns: max-content 1fr`
12-column	`repeat(12, minmax(0, 1fr))` + `grid-column: span 4`

Code to remember

```
.gallery { display: grid; gap: 1rem;
  grid-template-columns: repeat(auto-fill, minmax(min(200px, 100%), 1fr)); }
.page { display: grid; grid-template-columns: 200px minmax(0, 1fr) 200px;
  grid-template-areas: "header header header" "nav main aside" "footer footer footer"; }
.page > header { grid-area: header; }
.card { grid-row: span 3; display: grid; grid-template-rows: subgrid; }
```

In the interview

Q: Explain grid in 30 seconds. **A:** Two-dimensional; define tracks with lengths, `fr`, `minmax`, `repeat`; items auto-placed or placed by lines/spans/areas; extra items create implicit tracks; `justify-*` inline, `align-*` block; subgrid shares parent tracks.

Q: How is `fr` calculated? **A:** Container minus fixed tracks and gaps, divided by total `fr`; 700 − 200 − 40 = 460 → 153.33.

Q: auto-fill vs auto-fit? **A:** Same until too few items; auto-fill keeps empty tracks, auto-fit collapses them so items stretch.

Q: Why are my `1fr` columns unequal? **A:** `1fr` has an `auto` (min-content) minimum; use `minmax(0, 1fr)`.

Q: Responsive grid without media queries? **A:** `repeat(auto-fit, minmax(min(200px, 100%), 1fr))`.

Q: Grid or flexbox? **A:** Grid for page structure and uniform grids; flexbox for content-sized rows inside components; nest them.

Q: Reorder a layout at a breakpoint? **A:** Rewrite named areas in a media query, keeping DOM order sensible for keyboard/screen readers.

Q: What is subgrid for? **A:** Aligning the insides of sibling components to shared tracks.

Gotchas

- `%` columns + `gap` → horizontal scrollbar; use `fr`.
- auto-fill min larger than the container → overflow; `min(200px, 100%)`.
- Typo in an area name → implicit lines, item lands in a new track.
- Image doesn't fill its cell → aspect-ratio items align `start`; `width/height: 100%; object-fit: cover`.
- `justify-content` with `fr` tracks → no free space, no effect.
- Rows default to `auto` height.

TL;DR

`fr` shares what's left after fixed tracks and gaps; `minmax(0,1fr)` for equal columns; RAM = `repeat(auto-fill|fit, minmax(min(X,100%),1fr))`; fill keeps empty tracks, fit collapses them; grid for 2D, flex for 1D.

10. Responsive Design

Viewport meta and units, media queries and range syntax, mobile-first, user preferences, container queries, fluid type, intrinsic layouts, testing.

Must-know

- **Responsive** = fluid grids + flexible media + conditional CSS (`@media`, now `@container`). **Adaptive** = a few fixed-width layouts.
- **Mobile-first** — base CSS is the phone layout; `min-width` / `width >=` queries **add** features (progressive enhancement). Don't mix directions.
- **Viewport meta** — `width=device-width, initial-scale=1`; without it phones lay out at ~980px and shrink (measured scale 0.408 on a 400px phone). Never disable zoom (iOS ignores `user-scalable=no` since iOS 10). `viewport-fit=cover` + `env(safe-area-inset-*)` for notches.
- **Viewport units** — `svh` (toolbars shown), `lvh` (hidden), `dvh` (live, may relayout while scrolling); `vh` ≈ `lvh` on phones (why `100vh` overflows); `100vw` includes the scrollbar.
- **Media query anatomy** — type + features, combined with `and`, `,` (or), `not`. **Range syntax** `(640px <= width < 1024px)` removes boundary gaps. `em` in queries = browser default font size (not affected by `html` font-size). JS: `matchMedia(q)` + `change` event.
- **Source order** — a media query placed **before** its base rule loses (equal specificity).
- **Breakpoints** — where the content breaks, not device widths; prefer `em`.
- **`hover` / `pointer`** — `(pointer: coarse)` for bigger targets; don't use `(hover: hover)` alone to mean "desktop".
- **Dark mode** — `prefers-color-scheme` + `color-scheme: light dark` (themes form controls and scrollbars) + token overrides; `light-dark()` (Baseline 2024); user override via `[data-theme]`.
- **Container queries** — `container: card / inline-size` on a **wrapper**; `@container card (width >= 400px)` styles **descendants** only; `cqi` / `cqb` units (fall back to `svi` without a container). Size queries widely available since **Aug 2025**; style queries on custom properties Baseline **2026**; scroll-state queries Chromium only.

- **Fluid type** — `clamp(MIN, a·rem + b·vw, MAX)` ; slope = Δsize/Δviewport, intercept = size − slope × viewport; **keep a** `rem` **term** so user font settings and zoom work (vw-only stayed 16px when the default was 20px).
- **Intrinsic layouts** — `width: min(100% - 2rem, 60ch)` , flex-wrap sidebar, `repeat(auto-fit, minmax(min(X,100%),1fr))` — responsive without breakpoints.
- `image-set()` — resolution-switched CSS backgrounds (one file per DPR).

Key facts & numbers

Old	Range syntax
`(min-width: 640px)`	`(width >= 640px)`
`(max-width: 639.98px)`	`(width < 640px)`
both	`(640px <= width < 1024px)`

Preference feature	Baseline
`prefers-color-scheme`	widely available (2020)
`prefers-reduced-motion`	widely available (2020)
`prefers-contrast`	widely available (2022)
`forced-colors`	widely available (2022) — use system colours like `CanvasText`
`prefers-reduced-transparency`	**not** Baseline (Chromium only)
`prefers-reduced-data`	not supported anywhere

Media query for	Container query for
Page-level layout, user preferences, print, pointer, device	Components in different-sized slots, internal widget layout, `cqi` sizing

Test	Pass
Reflow (1.4.10 AA)	320 CSS px (= 1280 at 400%), vertical scroll only
Resize text (1.4.4 AA)	200% zoom, nothing clipped
Text spacing (1.4.12 AA)	line-height 1.5, paragraph 2em, letter 0.12em, word 0.16em
Target size (2.5.8 AA)	≥ 24×24 CSS px
No sideways scroll	`scrollWidth ≤ clientWidth` 320px → 1920px+

Code to remember

```
.layout { display: grid; gap: 1rem; }
@media (width >= 40em) { .layout { grid-template-columns: 1fr 1fr; } }
@media (width >= 64em) { .layout { grid-template-columns: repeat(3, 1fr); } }
.card-wrap { container: card / inline-size; }
@container card (width >= 400px) { .card { display: flex; } }
h1 { font-size: clamp(1.75rem, 1.2rem + 2.5vw, 3rem); }
@media (prefers-reduced-motion: reduce) { *, *::before, *::after { animation-duration: .01ms !important;
transition-duration: .01ms !important; } }
```

In the interview

Q: How do you build a responsive site? **A:** Mobile-first base + `min-width` queries; viewport meta; intrinsic flex/grid and `clamp()` for fewer breakpoints; container queries for components; respect preferences; `srcset` ; test at 320px and 200% zoom.

Q: Why does `100vh` misbehave on mobile? **A:** It's the large viewport (toolbars hidden), so the bottom is cut off when toolbars show; use `svh` / `dvh`.

Q: Media vs container queries? **A:** Viewport vs ancestor size; page layout vs component layout. You still need media queries for page layout and preferences.

Q: Why can't the container be `container-type: size` by default? **A:** Size containment ignores content, so height would collapse to 0; `inline-size` only fixes width.

Q: Why keep a `rem` term in `clamp()`? **A:** A vw-only size ignores the user's default font size and zoom (WCAG 1.4.4).

Q: How do you avoid horizontal scrolling? **A:** `max-width` not fixed widths, flexible media, `overflow-wrap: anywhere`, `minmax(0,1fr)`, automated check at 320px.

Gotchas

- Missing viewport tag → every phone breakpoint silently fails.
- `container-type` on the element you want to restyle → it can only restyle descendants.
- `inline-size` container inside a shrink-to-fit parent (flex `none`, inline-block, float) → collapses; give it width from outside.
- Unnamed `@container` → matches the nearest size container, maybe not the one you meant.
- `font-size: 4vw` alone → unreadable on phones, ignores settings. `clamp(16px, …, 24px)` → use `rem` limits.
- Fixed `width: 500px` → #1 overflow cause; use `max-width`.

TL;DR

Mobile-first + viewport meta + range queries by content; `svh` / `dvh` not `vh`; container queries for components; `clamp()` with a `rem` term; intrinsic layouts; test at 320px, 200% zoom, dark and reduced motion.

11. Typography, Colour and Visual Effects

Fonts and `@font-face`, `font-display`, loading, text properties, WCAG contrast, colour derivation, backgrounds, borders, shadows, filters, masks, `object-fit`.

Must-know

- **Font stack** — always end with a generic family (`system-ui`, `sans-serif`, `serif`, `monospace`); never quote generic names. A `system-ui` stack costs 0 bytes.
- **`@font-face`** + **WOFF2** (the only format you need). One **variable font** (`font-weight: 100 900`) replaces many static files; axes `wght`, `wdth`, `ital`, `slnt`, `opsz` — prefer the matching CSS properties. Missing faces are **synthesized** (faux bold) unless `font-synthesis: none`; asking for 600 with only 400/700 picks 700.
- **FOIT / FOUT** — invisible text vs fallback-then-swap (can shift layout).
- **Font loading** — preload 1–2 critical fonts **with** **`crossorigin`** (fonts are CORS-mode; without it the font downloads twice); self-host (no extra connection; GDPR ruling LG München I, Jan 2022); subset with `unicode-range`; match fallback metrics with `size-adjust` (reduces CLS; `ascent-override` / `descent-override` not in Safari); long cache + hashed names.
- **Unitless** **`line-height`** — a length is computed once and inherited (24px for 32px text); a number is re-multiplied per element. Body 1.4–1.6, headings 1.1–1.25.
- **Measure** — 45–75ch (`max-inline-size: 65ch`). `text-wrap: balance` (Baseline 2024) for headings; `pretty` not Baseline. `hyphens: auto` needs `lang`. `tabular-nums` for tables/prices.
- **Truncation** — single line: `white-space: nowrap; overflow: hidden; text-overflow: ellipsis`; multi-line: `-webkit-line-clamp`.
- **Contrast** — `(L1 + 0.05)/(L2 + 0.05)`; OKLCH/HSL lightness does **not** guarantee WCAG contrast — check the real pair. Never colour alone (link underlines: WCAG 1.4.1).
- **Deriving colours** — `color-mix(in oklch, var(--brand) 70%, black)`; relative colour syntax `oklch(from var(--c) l c h)` (Baseline 2024).

- **Backgrounds** — comma-separated layers, **first on top**; always a `background-color` fallback; `cover` fills and crops, `contain` fits; shorthand `url() center / cover no-repeat`; `background-attachment: fixed` ignored on iOS, not Baseline. Gradients: linear/radial/conic/repeating; `in oklch` avoids grey midpoints.
- **Focus = `outline`** — no layout shift, follows radius, **survives forced-colours mode** (box-shadow rings are removed there). Pattern: transparent outline + box-shadow ring.
- **Pills** — `border-radius: 999px`; circle = `50%` on a square.
- **Shadows** — `[inset] x y blur spread colour`, comma layers (first on top); `drop-shadow()` follows alpha; don't animate shadows directly.
- **Effects** — `filter` (element itself), `backdrop-filter` (behind; Baseline 2024), `mix-blend-mode`, `clip-path`, `mask-image`. They **create stacking contexts**; filters also become the containing block for `position: fixed`.
- **`object-fit`** — for `<img>` / `<video>` (replaced elements only) in a box with both dimensions: `fill` (distorts), `contain` (min scale, letterbox), `cover` (max scale, crop), `none`, `scale-down`; with `object-position`.

Key facts & numbers

font-display	Block	Swap	Use
`block`	short (≈3 s)	infinite	icon fonts
`swap`	≈0–100 ms	infinite	brand/heading fonts
`fallback`	extremely short	≈3 s	body text if it comes fast
`optional`	≈100 ms	**none**	stability first; used from cache next page

Contrast (WCAG 2.2)	Level	Min
1.4.3 normal text	AA	**4.5:1**
1.4.3 large text (≥24px or ≥18.66px bold)	AA	**3:1**
1.4.6 enhanced	AAA	7:1 (large 4.5:1)
1.4.11 non-text (icons, input borders, focus rings)	AA	**3:1**

Measured pair	Ratio
#767676 on white	lightest AA grey (4.54:1); #777777 fails
White on #3b82f6	3.68:1 — large/UI only
Yellow on white	1.44:1

	border	outline	box-shadow ring
Takes space	yes	no	no
In forced colours	yes	yes	**no**

Code to remember

```
@font-face { font-family: "Inter"; src: url(/fonts/inter.woff2) format("woff2");
  font-weight: 100 900; font-display: swap; unicode-range: U+0000-00FF; }
body { font: 1rem/1.5 Inter, system-ui, sans-serif; }
:focus-visible { outline: 3px solid #2563eb; outline-offset: 2px; }
.thumb { width: 100%; aspect-ratio: 16/9; object-fit: cover; }
```

```
<link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin>
```

In the interview

Q: How do you load web fonts well? **A:** `font-display` per role, self-host WOFF2, subset with `unicode-range`, preload 1–2 with `crossorigin`, one variable font, `size-adjust` fallback to cut CLS.

Q: Why does font preload need `crossorigin`? **A:** Fonts are fetched in CORS mode; otherwise the preload doesn't match and the font downloads twice.

Q: `optional` vs `swap`? **A:** `optional` when layout stability matters more than the brand font on first view; it's used from cache next time.

Q: Why unitless `line-height`? **A:** Lengths are computed once and inherited; numbers scale with each element's font size.

Q: How is contrast computed and what are the AA thresholds? **A:** `(L1+0.05)/(L2+0.05)` on relative luminance; 4.5:1 text, 3:1 large text and UI.

Q: Why outline for focus, not box-shadow? **A:** Outline takes no space and survives forced-colours mode.

Q: `cover` vs `contain`? **A:** Max scale and crop vs min scale and letterbox.

Gotchas

- Gradient layer listed after the image → underneath, invisible.
- Meaningful images as CSS backgrounds → no alt.
- `line-height` in px on `body` → every heading inherits fixed spacing.
- Centre-aligned long paragraphs; all-caps paragraphs.
- `object-fit` with only `width` set, or on a `<div>` → no effect.
- Filters on an ancestor → `position: fixed` children stop being viewport-fixed.

TL;DR

Generic fallback, WOFF2 variable font, right `font-display`, preload with `crossorigin`; unitless line-height, 45–75ch; 4.5:1 / 3:1 contrast; layers first-on-top; outline for focus; `object-fit: cover` for thumbnails.

12. Transitions, Transforms and Animation

Transitions and easing, 2D/3D transforms, keyframes, the Web Animations API, performance, `@starting-style`, reduced motion, view transitions, scroll-driven animations.

Must-know

- **Transition** — animates a *change* between two states: `transition: prop duration easing delay` (first time = duration, second = delay). Put it on the **base rule**, not `:hover`. Only interpolable values (not `auto`). `transitionend` fires per property. A reversed transition is shortened. UI feedback ≈ **100–300ms**, bigger moves up to ~500ms.
- **Easing** — `ease` (default), `ease-in` (leaving), `ease-out` (entering/responding — feels snappy), `ease-in-out` (between two on-screen positions), `cubic-bezier()` (x in 0–1, y may overshoot), `steps(n)` (jump-end by default), `linear(points)` for bounces/springs (Baseline since Dec 2023).
- **Transforms** — move the **painted** box, not the layout box. Order matters (read right to left). `%` in `translate` = the element's **own** size (`translate(-50%, -50%)` centres on a point). `transform-origin`. Individual `translate` / `rotate` / `scale` (Baseline 2022) apply in a fixed order and animate independently. Transforms create **stacking contexts** and **containing blocks** (for fixed descendants).
- **3D** — `perspective` on the **parent**, `transform-style: preserve-3d`, `backface-visibility: hidden` → flip cards. `overflow`, `opacity < 1`, `filter` flatten 3D.
- **Keyframes** — `@keyframes` + `animation: name duration easing delay count direction fill-mode`. Easing applies **per keyframe segment**. Negative delay starts part-way (stagger spinners). `animation-composition` Baseline 2023.
- **Transition vs animation** — transition: two states triggered by a change, reverses naturally. Animation: runs by itself, intermediate steps, iteration/direction choreography.

- **WAAPI** — `el.animate(keyframes, options)`, `pause()`, `currentTime`, `playbackRate`, `reverse()`, `await anim.finished`, `document.getAnimations()` (includes CSS animations).
- **Performance** — animate **`transform` and `opacity`** (compositor, GPU, survive a busy main thread) — not `left` / `width` (≈60 layouts per second, measured). **FLIP** for layout changes. `will-change` sparingly (too many layers → memory; also a stacking context).
- **Enter/exit** — `@starting-style` (Baseline 2024) gives the "from" style when an element appears or leaves `display: none`; `transition-behavior: allow-discrete` + `display` / `overlay` keep it visible while animating out (display transitions Chrome/Safari; `overlay` Chromium).
- **Reduced motion** — opt motion **in** with `@media (prefers-reduced-motion: no-preference)`; replace motion (fade) rather than removing feedback.
- **View transitions** — `document.startViewTransition(update)` + `view-transition-name` → snapshot, update DOM, morph via `::view-transition-group/old/new`.
- **Scroll-driven** — `animation-timeline: scroll()` / `view()` + `animation-range`; duration ignored; use `linear`. **Not Baseline** (no Firefox) → `@supports`, content visible without it.

Key facts & numbers

Changed property	Pipeline	Cost
`width`, `height`, `top` / `left`, `margin`, `padding`, `font-size`	layout → paint → composite	highest
`color`, `background`, `box-shadow`, `border-radius`	paint → composite	medium
`transform`, `opacity` (often `filter`)	composite only	lowest

animation-fill-mode	During delay	After end
`none`	own style	own style
`forwards`	own style	**last keyframe**
`backwards`	**first keyframe**	own style
`both`	first keyframe	last keyframe

WCAG motion	Level	Rule
2.2.2 Pause, Stop, Hide	A	Auto motion lasting **> 5 s** shown with other content must be pausable
2.3.1 Three Flashes	A	**≤ 3 flashes per second**
2.3.3 Animation from Interactions	AAA	Interaction motion can be disabled

Feature	Baseline status (Sept 2026)
Same-document view transitions	**Baseline newly available since Oct 2025** (Chrome 111, Safari 18, Firefox 144)
`view-transition-class`	Baseline since Oct 2025
View transition types	Baseline since Jan 2026
Cross-document (`@view-transition { navigation: auto }`)	**Not Baseline** (Chrome 126+, Safari 18.2+, no Firefox)
Scroll-driven animations	**Not Baseline** (no Firefox)
`@starting-style`	Baseline 2024

Code to remember

```
.btn { transition: translate 200ms ease-out, opacity 200ms; }
.btn:hover { translate: 0 -2px; }
.toast { transition: opacity .3s, display .3s allow-discrete; }
.toast[hidden] { opacity: 0; display: none; }
@starting-style { .toast { opacity: 0; } }
@media (prefers-reduced-motion: no-preference) { .hero { animation: rise .6s ease-out both; } }
```

```
if (!document.startViewTransition) update(); else document.startViewTransition(update);
```

In the interview

Q: Why animate `transform` / `opacity`? **A:** They skip layout and paint; the compositor moves an already painted layer on the GPU, even when the main thread is busy.

Q: How do you animate a layout change cheaply? **A:** FLIP — measure First and Last, Invert with a transform, Play the transform back to zero.

Q: Downside of `will-change` everywhere? **A:** Too many layers → memory pressure, can be slower; creates stacking contexts.

Q: Transition or keyframe animation? **A:** Transition for two-state changes that reverse; animation for self-running, multi-step or looping motion.

Q: Why doesn't my element animate in from `display: none`? **A:** No "before" style exists; use `@starting-style` plus `allow-discrete`.

Q: What does `animation-fill-mode: both` do? **A:** First keyframe applies during the delay, last keyframe after the end.

Q: How do you respect reduced motion? **A:** Opt motion in under `prefers-reduced-motion: no-preference`; replace, don't remove feedback; pause controls for > 5 s motion.

Gotchas

- `transition` only on `:hover` → instant change back.
- `transition: all` → animates layout properties you didn't intend.
- Expecting timing function to span the whole animation → it applies per segment.
- Transform order `rotate translate` ≠ `translate rotate`.
- Transformed/filtered ancestor → `position: fixed` breaks.
- Cross-document view transitions and scroll-driven animations are not Baseline → progressive enhancement only.

TL;DR

Transitions for state changes (on the base rule), keyframes for self-running motion; animate `transform` / `opacity`; `@starting-style` + `allow-discrete` for enter/exit; opt motion in for reduced-motion users; view transitions same-document Baseline Oct 2025.

13. Modern CSS and Architecture

Native nesting, `:is()` / `:where()`, `:has()`, `@layer`, `@scope`, logical properties, BEM, Tailwind v4, CSS Modules, CSS-in-JS, Sass/PostCSS, resets, theming.

Must-know

- **Native nesting** (Baseline 2023) — nested selectors are relative to the parent (`.a { .b {} }` = descendant `.a .b`; write `> .b` for child). `&` = parent, behaves like `:is()` **of the parent list** (takes its highest specificity). **No string joining** — Sass `&__title` doesn't work. `.x & {}` inside `.card` = a `.card` inside `.x`. Nested `@media` works. Keep 2–3 levels.
- `:is()` / `:where()` — "any of"; **forgiving** lists (unknown selectors ignored); `:is()` = max argument specificity, `:where()` = 0 → `:where()` for resets and library defaults. Pseudo-elements not allowed

inside.

- `:has()` (Baseline 2023) — parent / previous-sibling / state selector: `.card:has(img)`, `h3:has(+ .subtitle)`, `label:has(:checked)`, `html:has(dialog[open])`. No nesting `:has()` in `:has()`. **Anchor it** for performance (avoid `*:has()`); never for behaviour.
- `@layer` — declare order up front (first mention fixes position); later layers win **regardless of specificity** (a class in `components` beat `#app .btn.btn-primary` in `vendor`); unlayered beats layers; `!important` flips; `revert-layer`; `@import url(x) layer(vendor)` must precede other rules.
- **Recommended layer order** — `reset, tokens, vendor, base, layout, components, utilities, overrides`.
- `@scope` (Baseline March 2026) — `@scope (.card) to (.content)` = **donut scope**; `:scope`; **proximity** breaks specificity ties (nearest theme wins); root adds no specificity; doesn't block **inheritance**. Full two-way isolation needs Shadow DOM.
- **Logical properties** — `inline` / `block` + `start` / `end`: `margin-inline`, `padding-block-start`, `inset-inline-end`, `inline-size`, `text-align: start`, `border-start-start-radius`. One stylesheet for LTR, RTL and vertical text. Mirror directional icons with `:dir(rtl)` (Baseline 2023).
- **Why architecture** — global namespace, specificity/order dependence, no dead-code detection, implicit DOM coupling.
- **BEM** — `block__element--modifier`, flat single classes; needs discipline; still one global namespace.
- **Tailwind v4** — scans source, generates only used utilities into `@layer utilities` (28 classes → 28 rules, ~6.5 KB); CSS size plateaus. Class names must be **static strings** (`'text-' + color` isn't found → map to full names). `@apply` sparingly.
- **CSS Modules** — build renames `.title` → `.title_14He7q`; `composes`; zero runtime. Not the same as "CSS module scripts" (`import … with { type: 'css' }` — not Baseline, no Safari).
- **CSS-in-JS** — runtime: hash + inject during render (101 rules for 101 progress values); costs render time (INP/LCP), poor fit with React Server Components; styled-components in **maintenance mode** (March 2025). Prefer zero-runtime, Modules, Tailwind, or custom properties.
- **Sass** — variables, mixins, loops, `&__` joining; `@use` not `@import`. Native now covers variables, nesting, math (`calc/min/max/clamp`, `round()/mod()` 2024), colour maths (`color-mix()`, relative colours) — not mixins/loops; `@function` and `if()` are **Chromium only**.
- **Reset** — small modern reset in `@layer reset`: box-sizing, margins, media `max-width: 100%`, `font: inherit` on form controls, `overflow-wrap`; **never remove focus outlines**.
- **Theming** — primitive → semantic → component tokens; `color-scheme: light dark` + `light-dark()` (Baseline 2024); override by pinning `color-scheme` via `[data-theme]`; persist in localStorage and apply with an inline `<head>` script **before first paint** (no flash).

Key facts & numbers

	BEM	Tailwind	CSS Modules	Runtime CSS-in-JS	Zero-runtime
Scoping	convention	none needed	build rename	generated classes	build
Runtime cost	none	none	none	**yes**	none
Dead code	manual	automatic	per file	automatic	automatic
CSS size	grows	**plateaus**	grows	grows	grows / atomic plateaus

Tier	Example	Used by
Primitive	`--blue-600: #2563eb`	semantic tokens only
Semantic	`--color-surface`, `--color-text`	components
Component	`--card-bg: var(--color-surface)`	one component

Code to remember

```
@layer reset, tokens, vendor, base, components, utilities;
@import url(lib.css) layer(vendor);
.card { padding: 1rem;
  &:hover { outline: 2px solid; }
  @media (width >= 40em) { display: flex; } }
@scope (.card) to (.content) { img { border-radius: 8px; } }
:root { color-scheme: light dark;
  --color-surface: light-dark(#fff, #0f172a); --color-text: light-dark(#0f172a, #e2e8f0); }
:root[data-theme="dark"] { color-scheme: dark; }
```

In the interview

Q: Native nesting vs Sass? **A:** Native `&` acts like `:is()` of the parent (highest specificity), no string concatenation; relative selectors are descendants.

Q: Specificity of `.card:has(> img, #hero)`? **A:** (1,1,0) — `.card` plus the most specific argument.

Q: Override a library's high-specificity CSS without `!important`? **A:** Import it into a low layer; your later-layer or unlayered rules win.

Q: What does `@scope … to` do? **A:** Limits matching to a subtree with a lower boundary (donut scope); proximity breaks ties.

Q: How do you choose a CSS architecture? **A:** Content site → plain CSS with layers, nesting, BEM or `@scope`, tokens. Component app → Tailwind or CSS Modules + tokens. Avoid runtime CSS-in-JS for new work. Don't mix three approaches.

Q: Migrate a big legacy stylesheet? **A:** Wrap it in a low `@layer legacy`, build new components in a higher layer, delete legacy as pages migrate.

Q: How do you avoid the dark-mode flash? **A:** Apply the saved theme synchronously in `<head>` before first paint (or server-render from a cookie).

Q: Why does one `data-theme` attribute flip every token? **A:** `light-dark()` follows `color-scheme`, so pinning it on the root flips all semantic tokens.

Gotchas

- `light-dark()` without `color-scheme` → always picks light.
- Components using primitive tokens → dark mode needs component edits.
- Theme set in `useEffect` / `DOMContentLoaded` → flash.
- `!important` in a later layer expected to win → it's reversed.
- Own CSS unlayered while framework is layered → your stray rules beat utilities.
- `:is()` in library defaults with an ID argument → hard to override; use `:where()`.
- Utilities don't add focus styles — `focus-visible:ring-2` does.

TL;DR

Nesting (`&` = `:is(parent)`), `:where()` for zero-specificity defaults, `:has()` as parent selector, `@layer` beats specificity, `@scope` for donut scoping, logical properties for RTL; Tailwind or Modules for apps; tokens + `color-scheme` + `light-dark()` for themes.

14. Performance, SEO and Revision

The critical rendering path, render-blocking resources, Core Web Vitals (measured and fixed), SEO basics and JSON-LD.

Must-know

- **Critical rendering path** — minimum work before first pixels: HTML → DOM, CSS → CSSOM → render tree → layout → paint → composite. A resource is **critical** if the first render waits for it.
- **Render-blocking** — CSS in `<head>` blocks paint (else **FOUC**); classic scripts block parsing **and wait for pending CSS**. Images, fonts, `async` / `defer` scripts don't block render but can delay LCP.

- **Three levers** — fewer critical resources, fewer critical bytes, shorter critical path (round trips; avoid chains like CSS `@import`, JS-injected hero images).
- **Toolbox** — inline critical CSS (a few KB), load the rest non-blocking (non-matching `media` is non-blocking); `defer` in `<head>` (better than end of body — download starts earlier); `preconnect` to 1–3 key origins; preload only late-discovered critical resources; HTTP **103 Early Hints**.
- **Measured** (CSS 800 ms / JS 600 ms): all-blocking FCP 848 ms; `defer` only 844 ms (CSS was the bottleneck); inline critical CSS + non-blocking sheet + `defer` → **40 ms**.
- **LCP** — candidates: `<img>`, SVG `<image>`, `<video>` poster/first frame, CSS `background-image: url()`, block text. The **last** candidate before first input is the LCP. Sub-parts: **TTFB → resource load delay → load duration → element render delay**.
- **INP** — per interaction (click/tap, key press; not hover/scroll): **input delay + processing time + presentation delay**; page INP = worst (one outlier ignored per 50 interactions). **Replaced FID in March 2024.**
- **CLS** — shift score = **impact fraction × distance fraction**; shifts within 500 ms of input excluded (`hadRecentInput`); grouped into session windows (gaps < 1 s, ≤ 5 s); CLS = largest window's sum.
- **Field vs lab** — field (CrUX 28-day, Search Console, `web-vitals` RUM) is what Google uses; lab (Lighthouse, WebPageTest, DevTools) is debuggable; Lighthouse navigation mode uses **TBT** as INP proxy.
- **Long task** — main-thread work > **50 ms**; delays input.
- **SEO** — crawl → render (Google runs evergreen Chromium) → index → rank. `robots.txt` blocks **crawling, not indexing** (use `noindex` on a crawlable page); canonical is a strong **hint**, absolute URL; `hreflang` for translations; mobile-first indexing; **keywords meta ignored**; real `<a href>` with descriptive text; **JSON-LD** schema.org (Google's recommendation) = eligibility for rich results, not a guarantee or ranking boost. Core Web Vitals help as page experience, but relevance dominates.

Key facts & numbers

Core Web Vitals thresholds (75th percentile of real visits, mobile and desktop separately):

Metric	Measures	Good	Needs improvement	Poor
LCP	main content appeared	**≤ 2.5 s**	2.5–4.0 s	> 4.0 s
INP	response to clicks, taps, keys	**≤ 200 ms**	200–500 ms	> 500 ms
CLS	unexpected movement	**≤ 0.1**	0.1–0.25	> 0.25

Other metric	Good
TTFB	≤ 0.8 s
FCP	≤ 1.8 s

API / feature	Baseline status (Sept 2026)
LCP and Event Timing APIs	**Baseline newly available since Dec 2025** (Safari 26.2)
Layout Instability API (CLS)	**Chromium-only**
`rel="preload"`	widely available
`fetchpriority`	Baseline since Oct 2024
`content-visibility: auto`	Baseline since Sept 2025
`blocking="render"`	**not Baseline** (no Firefox)
Speculation rules	**Chromium-only**

Measured fix	Before → after
LCP / CLS / INP	2648 ms / 0.195 / 304 ms → 1844 ms / 0 / < 16 ms
How	hero `<img>` in HTML with `width` / `height` + `fetchpriority="high"`, reserved banner space, deferred script, yield in the click handler

Fix list	Actions
LCP	TTFB (CDN, caching, no redirects, SSR/SSG); load delay (image in HTML, `fetchpriority="high"`, never lazy, preload if in CSS); duration (`srcset`, AVIF/WebP, CDN); render delay (no blocking CSS/JS, don't hide content until JS)
INP	break long tasks; defer third parties; `await scheduler.yield()`; debounce; Web Worker; small DOM; no layout thrashing; `content-visibility: auto`
CLS	`width` / `height` or `aspect-ratio`; reserve space for ads/embeds/banners; nothing inserted above; font metrics (`size-adjust`) or `font-display: optional`; animate `transform`; keep bfcache eligibility

Other basics	Practice
Caching	fingerprinted files + `Cache-Control: max-age=31536000, immutable`
Budgets	e.g. LCP < 2.5 s on mid-range phone, JS < 200 KB compressed; Lighthouse CI

Code to remember

```
<link rel="preconnect" href="https://cdn.example">
<style>/* critical above-the-fold CSS */</style>
<link rel="stylesheet" href="rest.css" media="print" onload="this.media='all'">
<script src="app.js" defer></script>
<img src="hero.avif" alt="…" width="1200" height="600" fetchpriority="high">
<script type="application/ld+json">
{ "@context": "https://schema.org", "@type": "Article", "headline": "…", "datePublished": "2026-06-01" }
</script>
```

In the interview

Q: What are the Core Web Vitals and thresholds? **A:** LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1, at p75 of real visits; INP replaced FID in March 2024.

Q: What are render-blocking resources and fixes? **A:** Stylesheets block paint, sync scripts block parsing; critical CSS inline, rest non-blocking, `defer` / `async`, LCP image discoverable, preconnect, no `@import` chains.

Q: Why can't Lighthouse show INP on page load? **A:** INP needs real interactions; lab uses TBT or a scripted flow.

Q: Is a shift after a click counted? **A:** Not within 500 ms of input.

Q: Does a font swap count as CLS? **A:** Yes if text reflows and moves content; `size-adjust` or `font-display: optional`.

Q: Four LCP sub-parts? **A:** TTFB, resource load delay, load duration, element render delay.

Q: Does `robots.txt Disallow` keep a page out of the index? **A:** No — it stops crawling; use `noindex` on a crawlable page.

Q: What HTML matters for SEO? **A:** Title, description, canonical, lang, viewport, no stray `noindex`, headings, real links, alt, JSON-LD.

Gotchas

- Preloading everything → competes with truly critical requests.
- Font preload without `crossorigin` → downloaded twice.
- Inlining **all** CSS → no cross-page caching, bloated HTML.

- Staging `noindex` shipped to production.
- Blocking CSS/JS in `robots.txt` → Google can't render.
- Client-only rendered content pages; `<span onclick>` links; same title everywhere; canonical to the home page from every page.

TL;DR

CSS blocks paint, scripts block parsing; inline critical CSS, `defer`, make the hero discoverable. LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 at p75. SEO: title, description, canonical, `noindex` care, semantic HTML, JSON-LD.

Last-minute checklist

- [ ] Page skeleton from memory: doctype, `<html lang>`, charset first (1024 bytes), viewport (no zoom blocking), unique title, `defer` scripts.
- [ ] One `h1`, no skipped levels; landmarks (`header` / `nav` / `main` / `aside` / `footer`, named `section`); no outline algorithm.
- [ ] Accessible name order: `aria-labelledby` → `aria-label` → label/alt → content → title/placeholder.
- [ ] `tabindex` 0 / −1 only; `:focus-visible` outlines; `<dialog>` + `showModal()`; targets ≥ 24×24.
- [ ] Contrast: 4.5:1 text, 3:1 large text and UI, 7:1 AAA; `(L1+0.05)/(L2+0.05)`.
- [ ] Images: `width` / `height`, purposeful alt, `srcset` + `sizes`, never lazy-load the LCP image, `fetchpriority="high"`.
- [ ] Forms: named + enabled + checked are sent; `type="button"` for script buttons; `:user-invalid`; `setCustomValidity('')`; server validates.
- [ ] Specificity: (A, B, C), no carrying, `:where` = 0, `:is/:not/:has` = max argument, `.a.a` counts twice.
- [ ] Cascade: origin & importance → context → inline → layers → specificity → order; unlayered beats layers; `!important` flips origins and layers.
- [ ] `inherit` / `initial` / `unset` / `revert` / `revert-layer`.
- [ ] `box-sizing: border-box` on `*, *::before, *::after`; % padding uses containing block **width**.
- [ ] Margin collapse rules (30/20 → 30; 30/−10 → 20; −10/−25 → −25); `flow-root` for a BFC.
- [ ] Stacking contexts: z-index on positioned, opacity < 1, transform, filter, `isolation: isolate`, fixed/sticky; top layer beats all.
- [ ] Flex: `basis + free × grow/Σgrow`; shrink by `shrink × basis`; `flex: 1` = `1 1 0%`, `auto` = `1 1 auto`, `none` = `0 0 auto`; `min-width: 0`.
- [ ] Grid: `fr` = (container − fixed − gaps)/Σfr; `1fr` = `minmax(auto, 1fr)` → `minmax(0, 1fr)`; n = floor((W + gap)/(min + gap)); fill keeps, fit collapses.
- [ ] Responsive: mobile-first `(width >= …)`; `dvh` / `svh` not `vh`; `100vw` includes scrollbar; container queries on a wrapper; `clamp()` with a `rem` term.
- [ ] Fonts: WOFF2, `font-display`, preload with `crossorigin`, unitless `line-height`, 45–75ch.
- [ ] Motion: animate `transform` / `opacity`; FLIP; `@starting-style` + `allow-discrete`; opt in under `prefers-reduced-motion: no-preference`.
- [ ] Modern CSS: nesting `&` = `:is(parent)`; `:has()` parent selector; `@layer` order; `@scope` donut; logical properties.
- [ ] Web Vitals: LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 at p75; long task > 50 ms; LCP sub-parts TTFB / load delay / load duration / render delay.
- [ ] SEO: title, description, canonical (hint), `noindex` vs `robots.txt`, real links, alt, JSON-LD.
- [ ] Baseline to quote: `:has()` 2023 (wide June 2026); `@layer` 2022; nesting 2023; subgrid Sept 2023; container size queries wide Aug 2025; `@scope` March 2026; same-document view transitions Oct 2025;

scroll-driven animations and cross-document view transitions **not** Baseline; masonry Safari-only.